

Reinforcement Learning

Clase 05

Ignacio Brevis

`ignaciobrve@unisabana.edu.co`

Universidad de La Sabana

16 septiembre 2026



Bellman equation (forma matricial)

$$\star \quad \boxed{v_\pi = r_\pi + \gamma P_\pi v_\pi}$$

Función de **estado-valor** y función de **acción-valor** (para la política π)

$$\star \quad \boxed{v_\pi(s) = \mathbb{E}[G_t | S_t = s]} \quad \star \quad \boxed{q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]}$$

Relación entre estado-valor y acción-valor

$$\star \quad \boxed{v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a)}$$

Función de acción-valor (para la política π)

$$\star \quad \boxed{q_\pi(s, a) = \sum_{r \in \mathcal{R}} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')}$$

Una política π^* es **óptima** si su función de valor cumple que:

$$\star \quad \boxed{v_{\pi^*}(s) \geq v_{\pi}(s), \quad \underline{\forall s \in \mathcal{S}} \quad \underline{\forall \pi \in \Pi}}$$

Ecuación de optimalidad de Bellman (BOE) para cada estado

$$\star \quad \boxed{v_{\pi}(s) = \underline{\max_{\pi(s)}} \sum_{a \in \mathcal{A}} \pi(a|s) q_{\pi}(s, a)}$$

Ecuación de optimalidad de Bellman (BOE) vectorial

$$\star \quad \boxed{v^* = \max_{\pi \in \Pi} (r_{\pi} + \gamma P_{\pi} v^*)}$$

Obs: La BOE nos permite encontrar políticas óptimas y valores de estados óptimos

- Se dice que $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ es una **función contractiva** ✓ si existe $\eta \in (0, 1)$, tal que:

$$\|f(x_1) - f(x_2)\| \leq \eta \|x_1 - x_2\|,$$

- Se dice que $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ tiene un **punto fijo** ✓, si existe x^* , tal que:

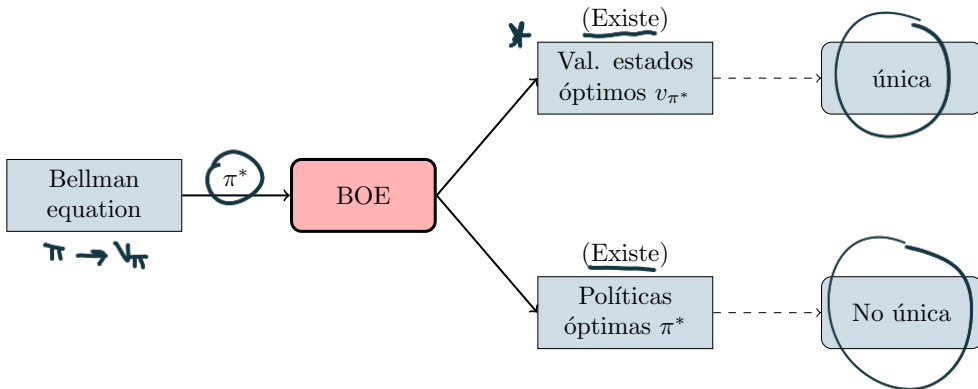
$$f(x^*) = x^*.$$

Si f es una función contractiva, entonces:

- Existe** un **único** punto fijo x^* .
- El punto fijo puede **calcularse iterativamente** como $x_{k+1} = f(x_k)$ (para cualquier punto inicial x_0) y $x_k \rightarrow x^*$ cuando $k \rightarrow \infty$.

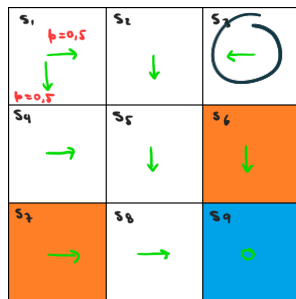
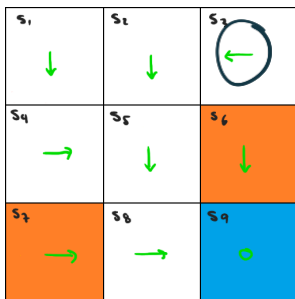
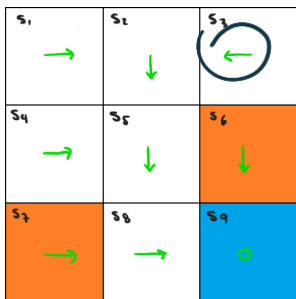
El lado derecho de la BOE $f(v_k) = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v_k)$ es una función contractiva.

✱ **La ecuación optimal de Bellman se puede resolver usando el teorema de la función contractiva!**



* Obs: Siempre existe una política óptima determinista (siguiendo un enfoque greedy)

$$\underline{\pi(a|s)} = \begin{cases} 1, & a = a_k^*(s), \\ 0, & a \neq a_k^*(s), \end{cases} \quad \text{con} \quad \underline{a^*(s)} \leftarrow \underbrace{\arg \max_a q^*(a, s)} \quad [\text{puede ser no único}]$$



Tomando $\gamma = 0.9$ (estas políticas no son optimas)

$$v_{\pi} = \begin{bmatrix} 7.29 & 8.10 & 7.29 \\ 8.10 & 9.00 & 10.00 \\ 9.00 & 10.00 & 10.00 \end{bmatrix}$$

$$V_{\pi^*}(s) \geq V_{\pi}(s)$$

Ejercicio: Calcular la función de valor para estas tres políticas (pueden usar el código `Example_01_value_function.ipynb` en GitHub)

La **evaluación de una política** π , como su nombre lo indica, significa calcular la función de valor v_π para dicha política.

Dado un valor inicial para v_0 (normalmente $v_0(s) = 0$ para todo s),

$$\star \quad v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \underbrace{\left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_k(s') \right]}_{q_k(s, a)}$$

(o de forma vectorial $v_{k+1} = (r_\pi + \gamma P_\pi v_k)$).

A medida que k aumenta, generamos una sucesión $(v_1, v_2, v_3, \dots, v_k, \dots)$ que converge a la función de valor v_π .

$$\boxed{v_k(s) \rightarrow v_\pi(s)}$$

Obs: Ya hicimos esto en la parte final de `Example_01_value_function.ipynb` $\star$

Policy iteration

El método de **iteración de política** nos permite calcular **políticas óptimas** usando **evaluación de política** y un **paso de mejora**.

Dada una política inicial π_0 (normalmente aleatoria), calculamos v_{π_0} usando la evaluación de política

$$* \rightarrow v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi_0(a|s) \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_k(s') \right] \quad \text{con } k = 1, 2, 3, \dots$$

Una vez que tenemos v_{π_0} , podemos mejorar la política por medio de

$$\pi_1(s) \leftarrow \arg \max_a \underbrace{\left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \sum_{s' \in \mathcal{S}} p(s'|s, a)v_{\pi_0}(s') \right]}_{q_{v_0}(s, a)}$$

(y luego iteramos sobre la política...)

Uno de los problemas de **policy iteration** es que en cada iteración de la política, requiere una **evaluación de política**, que es un proceso iterativo en si mismo.

El método de **iteración de valor** nos permite calcular **políticas óptimas** directamente resolviendo la **ecuación optimal de Bellman**.

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left[\sum_{r \in \mathcal{R}} p(r|s, a)r + \underbrace{\sum_{s' \in \mathcal{S}} p(s'|s, a)v_k(s')}_{q_k(s, a)} \right] \quad \text{con } k = 1, 2, 3, \dots$$

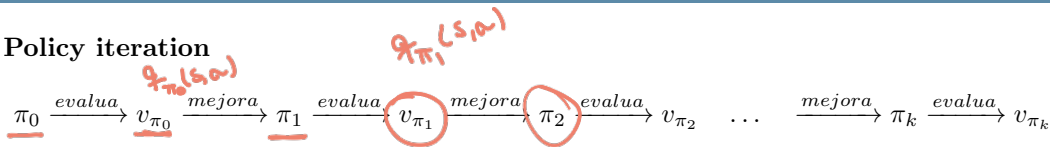
$V_0(s) = 0$
 $V(s) = \max_a \sum \pi(a|s) q(s, a)$

Una vez que tenemos la función de valor óptima v^* , podemos obtener (una) política óptima de la forma:

$$\pi^*(a|s) = \begin{cases} 1, & a = a^*(s), \\ 0, & a \neq a^*(s), \end{cases} \quad \text{con } a^*(s) \leftarrow \arg \max_a q^*(a, s) \quad [\text{puede ser no único}]$$

Policy iteration vs value iteration

* Policy iteration

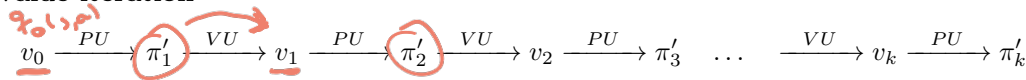


evalua: evaluación de política

mejora: mejora de política

* cuando la política es estable, por ejemplo $\pi_k = \pi_{k-1}$, paramos el algoritmo y $\pi^* \approx \pi_k$.

Value iteration



PU: actualización de política (policy update)

VU: actualización de valor (value update)

* cuando la función de valor es estable, por ejemplo $\|v_k - v_{k-1}\|$ es pequeño, paramos el algoritmo y $\pi^* \approx \pi'_k$.

Policy iteration vs value iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$ $V_{k+1} = r + \gamma P V_k$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma \underline{V(s')}]$ (greedy)

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Policy iteration vs value iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

| $\Delta \leftarrow 0$

| Loop for each $s \in \mathcal{S}$:

| $v \leftarrow V(s)$

| $\star V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$\star \pi(s) = \arg\max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

Policy iteration vs value iteration

Tarea: Encontrar una política óptima para el entorno gridworld de 3×3 (siguiendo las reglas vistas en la primera clase) por medio de los métodos:

- iteración de política
- iteración de valor

(tomar $\gamma = 0.9$)

Ver códigos:

`Lesson04/Example_03_policy_iteration.ipynb`

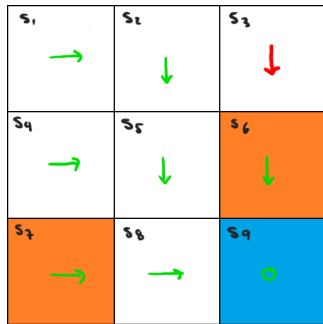
`Lesson04/Example_03_value_iteration.ipynb`

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

Policy iteration vs value iteration

$$v_{\pi^*} = \begin{bmatrix} 7.29 & 8.10 & 8.00 \\ 8.10 & 9.00 & 10.00 \\ 9.00 & 10.00 & 10.00 \end{bmatrix}$$

Handwritten purple "7.29" above the top-left element and a purple circle around the top-right element "8.00".



Ver códigos:

`Lesson04/Example_03_policy_iteration.ipynb`

`Lesson04/Example_03_value_iteration.ipynb`

Un **Proceso de decisión de Markov** (MDP) es un proceso estocástico discreto compuesto por:

- Conjuntos:
 - Espacio de estados $\mathcal{S}$ (state space)
 - Espacio de acciones $\mathcal{A}(s)$ (action space), con $s \in \mathcal{S}$
 - Conjunto de recompensas $\mathcal{R}(s, a)$ (reward set), con $s \in \mathcal{S}$ y $a \in \mathcal{A}(s)$

• **Modelo** o dinámica

- Probabilidad de transición de estado: $p(s'|s, a)$
- Probabilidad de recompensa: $p(r|s, a)$

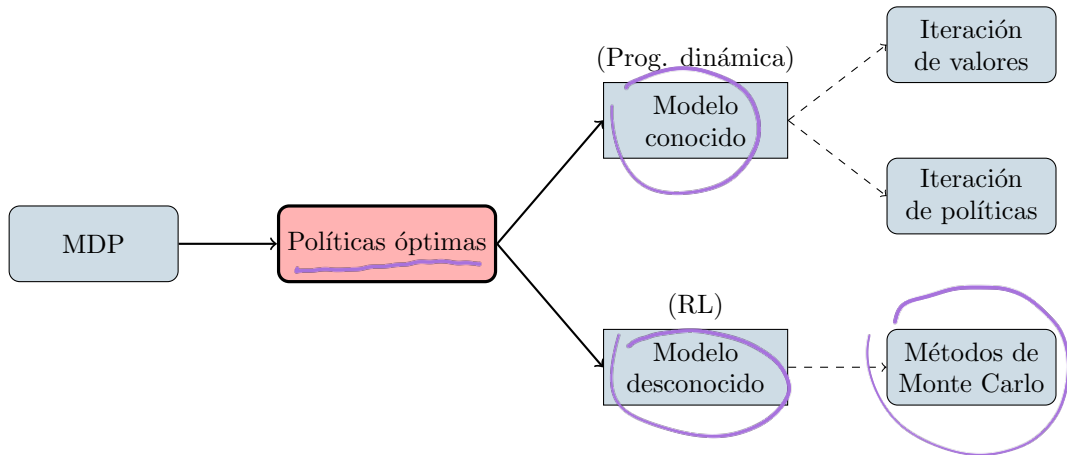
• Política

- Probabilidad de acción a : $\pi(a|s)$

• Propiedad Markoviana

- $p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t)$
- $p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t)$

Políticas óptimas sin modelo



Monte Carlo methods

- Fue creado por el matemático polaco Stanisław Ulam (13 de abril 1909 – 13 de mayo 1984)
- Ulam inventó el método de MC mientras participaba del Proyecto Manhattan en el laboratorio secreto de Los Alamos
- En 2000, el método de MC fue incluido en una lista de las 10 ideas más importantes del siglo 20 (en matemáticas aplicadas y ciencias de la computación)
- El nombre viene del casino Monte Carlo en Mónaco
- Tiene diversas aplicaciones en física, matemáticas e ingeniería: integración numérica, optimización, sistemas estocásticos, inferencia bayesiana, etc.

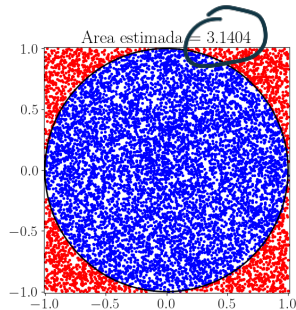
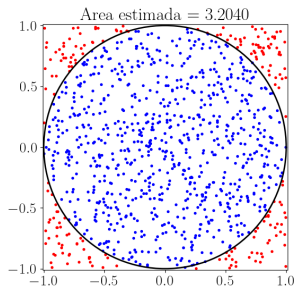
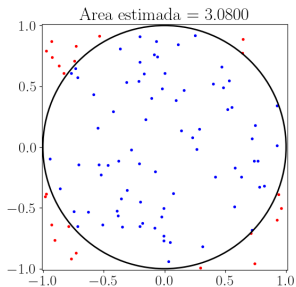


Image from https://en.wikipedia.org/wiki/Stanisław_Ulam.

Ejemplo: Cálculo de áreas

$$\frac{\#P_c}{\#P_T} = \frac{A_c}{A_{\square}} \Rightarrow A_c = \frac{\#P_c}{\#P_T} A_{\square}$$

$$A = \pi r^2 = \pi$$

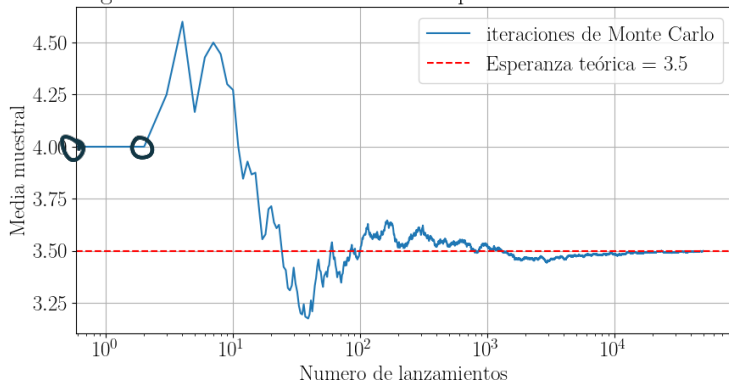


Ejemplo: Estimación del valor esperado

$$p(X=i) = \frac{1}{6} \quad i=1,2,\dots,6$$

$$\overset{3.5}{\mathbb{E}(X)} = \sum_x p(x)x \approx \frac{1}{N} \sum_{i=1}^N x_i$$

Convergencia del método de Monte Carlo para el lanzamiento de un dado



x_i samples

- **Usando modelo:** Conocemos la distribución de probabilidad

$$\star \mathbb{E}[X] = \sum_x p(X = x)x$$

- **Sin modelo:** No conocemos la distribución de probabilidad. Podemos aproximar la esperanza con un muestreo u observaciones $\{x_1, x_2, \dots, x_n\}$ de X .

$$\star \mathbb{E}[X] \approx \bar{x} := \frac{1}{n} \sum_{j=1}^n x_j$$

Ley de los grandes números

Dada una variable aleatoria X , supongamos que tenemos muestras i.i.d. $\{X_1, X_2, \dots, X_n\}$. Sea $\bar{X} = \frac{1}{n} \sum_{j=1}^n X_j$ el promedio de las muestras. Entonces,

$$\boxed{\mathbb{E}[\bar{X}] = \mathbb{E}[X]} \quad \text{y} \quad \boxed{\text{var}[\bar{X}] = \frac{1}{n} \text{var}[X]}$$

- Los métodos (o estimaciones) de Monte Carlo se refieren a una clase de técnicas que usan repeticiones de muestreo aleatorio para aproximar soluciones de problemas.
- Los métodos de Monte Carlo no requieren conocer el modelo.
- Esto es importante ya que las funciones de estado-valor y de acción-valor se definen como valores esperados!

$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$$

$$q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

Tomemos la **iteración de política (policy iteration)**:

$$\begin{cases} \text{Evaluación de política : } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{Mejora de política : } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left[\sum_r p(r|s,a) r + \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right]}_{\text{require model}}$$
$$= \arg \max_{\pi} \sum_a \pi(a|s) \boxed{q_{\pi_k}(s,a)}$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

$$\mathbb{E}(G_t | S_t=s, A_t=a) \text{ MC!}$$

- Requiere modelo:

$$\ast q_{\pi_k} = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

- No requiere modelo:

$$\ast q_{\pi_k} = \mathbb{E}[G_t|S_t = s, A_t = a]$$

$$\text{con } \underline{G_t} = \underline{R_{t+1}} + \gamma \underline{R_{t+2}} + \gamma^2 \underline{R_{t+3}} + \dots$$

Estimación de MC

Supongamos que realizamos n episodios (partiendo en s y eligiendo la acción a). El retorno de cada episodio es $\underline{g_{\pi_k}^{(i)}(s, a)}$. Entonces, $\underline{q_{\pi_k}(s, a)}$ se puede aproximar como:

$$q_{\pi_k} = \mathbb{E}[G_t|S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a)$$

Primer método: MC basic

La idea fundamental en los métodos de MC para reinforcement learning es usar una estimación model-free de la función de valor de acción (q values), por medio de

$$q_{\pi_k} = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a) \quad \leftarrow$$

Dada una política inicial π_0 , el algoritmo, en la k -ésima iteración:

π_k ① **Evaluación de la política:** Estimamos q_{π_k} para cada (s, a) . Específicamente, para cada par (s, a) obtenemos “suficientes” episodios para aproximar $q_{\pi_k}(s, a)$ por medio de $q_k(s, a)$.

② **Mejora de la política:** Actualizamos la política por medio de

$$\pi_{k+1}(s) \leftarrow \arg \max_{\pi} \pi(a|s) q_k(s, a),$$

para cada $s \in \mathcal{S}$. La política optimal (greedy) is $\pi_{k+1}(a_k^*|s) = 1$ con $a_k^* \in \arg \max_a q_k(a, s)$.

Algorithm 5.1: MC Basic (a model-free variant of policy iteration)

Initialization: Initial guess π_0 .

Goal: Search for an optimal policy.

For the k th iteration ($k = 0, 1, 2, \dots$), do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

✦ Collect sufficiently many episodes starting from (s, a) by following π_k

Policy evaluation:

✦ $q_{\pi_k}(s, a) \approx q_k(s, a)$ = the average return of all the episodes starting from (s, a)

Policy improvement:

➔ $a_k^*(s) = \arg \max_a q_k(s, a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

$$\underline{a_k^*(s)} = \arg \max_a q_k(s, a)$$

modelo datos
 ↓ ↓
samples experiences
(Estad) (RL)

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^* \\ 0 & a \neq a_k^* \end{cases} \quad \begin{matrix} 0,9 \\ \text{(greedy)} \end{matrix}$$

Primer método: MC basic

π_1

s_1 ↓	s_2 ↖	s_3 ↗
s_4 →	s_5 ↓	s_6 ↓
s_7 →	s_8 →	s_9 ○

$$9 \times 5 = 15 \quad (s, a)$$

$q(s_1, \uparrow)$
 $q(s_1, \rightarrow)$
 $q(s_1, \downarrow)$
 $q(s_1, \leftarrow)$
 $q(s_1, \circ)$

q	a_1	a_2	a_5
s_1			
s_2			
s_3			
s_4			
s_5			
s_6			
s_7			
s_8			
s_9			

$s_1 \xrightarrow{\uparrow} s_2 \xrightarrow{\uparrow} s_3 \xrightarrow{\rightarrow} s_2 \xrightarrow{\downarrow} s_5 \rightarrow \dots$
 $s_1 \xrightarrow{\uparrow} s_1 \xrightarrow{\rightarrow} s_2 \xrightarrow{\leftarrow} s_1 \xrightarrow{\uparrow} s_1 \rightarrow \dots$

$g_{\pi}^{(1)}$
 $g_{\pi}^{(2)}$

Obs: MC basic es una variante de el algoritmo de iteración de política.

Obs: En la práctica, no podemos calcular episodios infinitamente largos

$$\underline{g_{\pi_k}^{(i)}(s, a)} = r_0^{(i)} + \gamma r_1^{(i)} + \gamma^2 r_2^{(i)} + \gamma^3 r_3^{(i)} + \dots$$

Podemos calcular T “pasos”. Cuando T es muy pequeño, la política (y el valor de estado) no serán óptimos.

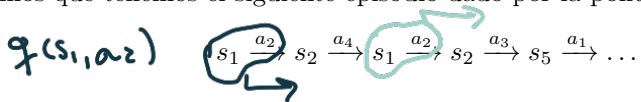
Obs: MC basic nos sirve para comprender la idea central detras de los métodos de MC en reinforcement learning, pero en la práctica no se utiliza (en esta forma) debido a su baja eficiencia.

Segundo método: MC exploring starts

Exploring Starts idea: Asegurar que tenemos suficientes episodios partiendo de cada par estado-acción.

¿Cómo podemos aprovechar mejor los episodios?

Supongamos que tenemos el siguiente episodio dado por la política π :

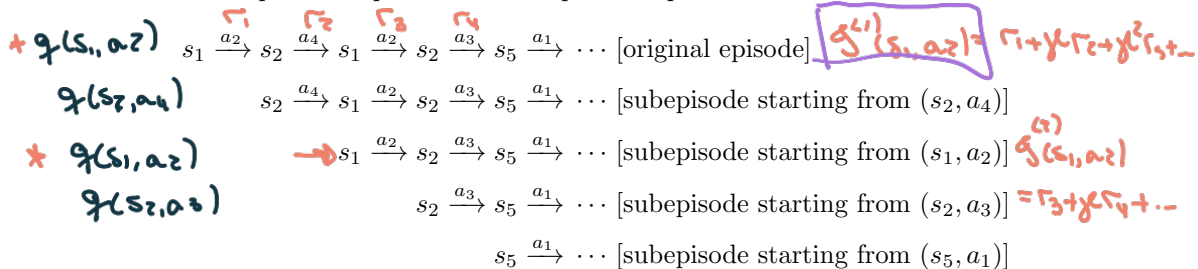


- Cada vez que un par estado-acción aparece en el episodio, lo llamamos **visita** de este estado-acción (s, a) .
- La estrategia *initial-visit* sólo estima el valor del par estado-acción inicial del episodio

El algoritmo MC Basic utiliza la estrategia *initial-visit*. Sin embargo, esto no es eficiente ya que el episodio visita muchos otros pares estado-acción

Segundo método: MC exploring starts

Idea 1: Descomponer el episodio en múltiples subepisodios!



- Estos nuevos ^{sub}episodios se pueden usar para estimar más valores de estado-acción.
- Un par (s, a) puede ser visitado varias veces en el mismo episodio.
- Si sólo contamos la primera visita, la estrategia se conoce como first-visit.
- Si contamos cada visita, la estrategia se conoce como every-visit.

Segundo método: MC exploring starts

Cuándo actualizar para política?

- En el primer método, la evaluación de la política ocurre una vez que todos los episodios fueron realizados. El agente debe esperar a que todos los episodios se hayan completado para actualizar la política (se gasta tiempo y no es eficiente).
- Otra alternativa, usar el retorno de un solo episodio para estimar la función de acción-valor. Por lo que podemos mejorar la política episodio por episodio.

Idea 2: Actualizar la política mientras calculamos retornos!

Obs: Dado que el resultado de un solo episodio no puede aproximar con precisión el valor de acción correspondiente, uno podría preguntarse si la segunda estrategia es adecuada. De hecho, esta estrategia se enmarca dentro de las **generalized policy iteration (GPI)**. Donde podemos seguir actualizando la política incluso si la estimación del valor no es lo suficientemente precisa.

Segundo método: MC exploring starts

Algorithm 5.2: MC Exploring Starts (an efficient variant of MC Basic)

Initialization: Initial policy $\pi_0(a|s)$ and initial value $q(s, a)$ for all (s, a) . $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$ for all (s, a) .

Goal: Search for an optimal policy.

For each episode, do

➡ *Episode generation:* Select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected (this is the exploring-starts condition). Following the current policy, generate an episode of length T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Initialization for each episode: $g \leftarrow 0$

For each step of the episode, $t = T-1, T-2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

➡ $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$

$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$

Policy evaluation:

➡ $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$

Policy improvement:

➡ $\pi(a|s_t) = 1$ if $a = \arg \max_a q(s_t, a)$ and $\pi(a|s_t) = 0$ otherwise

$$\begin{aligned} & s_0, a_0, r_1, s_1, a_1, \dots, a_{T-1}, r_T \\ & \quad \underbrace{\hspace{1.5cm}}_g \\ & \quad \underbrace{r + \gamma g}_{\tilde{g}} \\ & \quad \underbrace{\hspace{1.5cm}}_{r + \gamma \tilde{g}} \end{aligned}$$

$$\pi(a|s) = \begin{cases} 1, & a = a_k^*(s), \\ 0, & a \neq a_k^*(s), \end{cases} \quad \text{con} \quad a^*(s) \in \arg \max_a q^*(a, s) \quad [\text{política greedy}]$$

Soft policies

Llamaremos a una política soft si cada acción tiene una probabilidad positiva. Dado $s \in \mathcal{S}$,

$$\pi(a|s) > 0, \text{ para cada } a \in \mathcal{A}(s)$$

- Con políticas soft, unos pocos episodios suficientemente largos pueden cubrir todos los pares estado-acción.
- Elimina la necesidad de tener un gran número de episodios partiendo de cada par estado-acción. (requerido en *Exploring-starts*).

En particular, nos interesaremos en la política ε -greedy que nos permite la **exploración y explotación**.

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{para la acción greedy} \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{para las otras acciones} \end{cases}$$

donde $\varepsilon \in (0, 1)$ y $|\mathcal{A}(s_t)|$ representa el número de acciones posibles en el estado s .

Mejora de política: Elegimos a^* de la forma

$$a^* \in \arg \max_{\pi} q_{\pi_k}(s, a)$$

Pólítica greedy

$$\pi(a|s) = \begin{cases} 1, & a = a^*, \\ 0, & a \neq a^*, \end{cases}$$

(full explotación)

Pólítica ε -greedy

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & a = a^*, \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & a \neq a^*, \end{cases}$$

(exploración y explotación)

Algorithm 5.3: MC ϵ -Greedy (a variant of MC Exploring Starts)

Initialization: Initial policy $\pi_0(a|s)$ and initial value $q(s, a)$ for all (s, a) . $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$ for all (s, a) . $\epsilon \in (0, 1]$

Goal: Search for an optimal policy.

For each episode, do

Episode generation: Select a starting state-action pair (s_0, a_0) (the exploring starts condition is not required). Following the current policy, generate an episode of length

T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Initialization for each episode: $g \leftarrow 0$

For each step of the episode, $t = T - 1, T - 2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

$\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$

$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$

Policy evaluation:

$q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$

Policy improvement:

Let $a^* = \arg \max_a q(s_t, a)$ and

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$$